

Project Final Report: Out-of-Domain Face Recognition

Hyun-woo Seo
KAIST EE40034

Abstract

This report summarizes the experimental process and results for the out-of-domain face recognition project. The goal was to train a model on a Caucasian-dominant dataset (Train1) and adapt it to a Korean mixed-gender dataset (Train2) while adhering to a parameter limit of 11.5M. I explored various loss functions, hyperparameter configurations, model architectures, and data augmentation strategies. My experiments demonstrated that a transfer learning strategy—pre-training on the large out-of-domain dataset followed by fine-tuning on the in-domain dataset—yielded the best performance. Among the tested architectures, **MobileNetV2** trained with Softmax loss achieved the best Equal Error Rate (EER) of **9.78%** on the validation set. This result highlights the effectiveness of lightweight architectures combined with robust pre-training in constrained domain adaptation tasks.

1 Introduction

The primary challenge of this project is addressing the demographic bias between training data (Caucasian males) and test data (Korean mixed gender). Specifically, I utilized two training sets: **train1** (981k images, 2,882 IDs, out-of-domain) and **train2** (64k images, 949 IDs, in-domain). The objective was to minimize the EER on the validation set under strict constraints, including a model size limit (under 11.5M parameters) and a prohibition on external data.

2 Design Choices

I defined the search space for key design variables to optimize performance under the project constraints.

2.1 Model Architecture

To find the optimal backbone within the 11.5M parameter limit (excluding the FC layer), I explored the following candidates:

- **Base Model:** ResNet18 (11.4M)
- **Variants:** ResNet34 (Custom, 10.58M), ResNet18-SE-Net (11.26M), ResNet18-CBAM (11.26M)
- **Lightweight Models:** EfficientNet-B0 (4.66M), MobileNetV2 (2.22M)

Note that the standard ResNet34 exceeds the parameter limit. Therefore, I implemented a custom version where I adjusted the channel multipliers to reduce the number of channels per layer, ensuring it fits within the constraint.

2.2 Loss Functions

I investigated three categories of loss functions:

- **Classification:** Softmax Loss
- **Metric Learning:** Triplet Loss (Random/Hard Mining)
- **Angular Margin:** ArcFace Loss, Combined Loss

2.3 Implementation Details

All models were implemented in PyTorch. The default input size was 224×224 with standard normalization.

- **Optimizer:** Adam was selected as the default optimizer after comparing with SGD and AdamW. The weight decay was set to 0.
- **Scheduler:** StepLR with a decay factor of 0.9 was used to gradually reduce the learning rate.
- **Batch Size:** A batch size of 100 was found to be optimal for stability and convergence speed.
- **Embedding Dimension:** Fixed to 512.

3 Experiments & Ablations

All experiments were conducted using the provided server infrastructure.

3.1 Phase 1: Strategy & Baseline

In Phase 1, I established the baseline performance for different data combinations and loss functions.

Table 1: Phase 1 Results: Data & Strategies

Method	Data	Strategy	Best EER
Softmax	train1	Baseline	19.78%
Softmax	train2	Baseline	16.26%
Softmax	T1+T2	Baseline	17.22%
Triplet	train1	Random Mining	26.40%
Triplet	train2	Random Mining	24.40%
Triplet	T1+T2	Random Mining	26.64%
Softmax	T1 → T2	Fine-tuning (FT)	10.22%
Triplet	T1 → T2	Fine-tuning (FT)	17.16%

In-depth Analysis: The results highlight the critical importance of domain adaptation. Training solely on the large **train1** dataset resulted in poor performance (19.78%) due to the domain gap. Conversely, **train2**, despite being smaller, yielded better results (16.26%) because it matches the target domain. However, the

Transfer Learning strategy (T1 → T2) combined the strengths of both: learning general facial features from large-scale data and adapting to the specific demographics. This yielded a drastic improvement to **10.22% EER**.

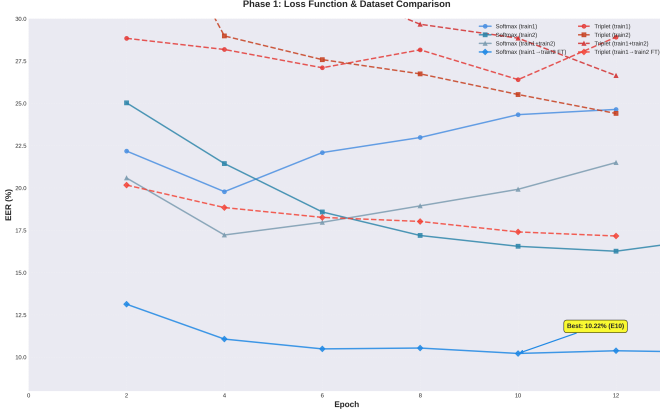


Figure 1: Comparison of validation EER across different training strategies. Transfer learning significantly outperforms single-dataset training.

3.2 Phase 2: Loss Function Ablation

I tested advanced loss functions (ArcFace, Triplet Hard) to see if they could surpass the Softmax baseline.

Table 2: Phase 2 Results: Advanced Loss Functions

Loss Function	Data	Details	Best EER
ArcFace	train2	s=64, m=0.3	33.53%
ArcFace	train2	s=30, m=0.3	35.53%
ArcFace	train2	s=64, m=0.5	40.41%
ArcFace	train2	s=30, m=0.5	51.88%
ArcFace	train1	Pre-train	22.33%
ArcFace	T1→T2	Fine-tuning	13.29%
Triplet Hard	train2	Margin=0.2	36.93%
Combined (0.7)	train1	Pre-train	23.87%
Combined (0.7:0.3)	T1→T2	FT (Softmax:Arc)	11.17%
Combined (0.5)	train1	Pre-train	23.73%
Combined (0.5:0.5)	T1→T2	FT (Softmax:Arc)	11.43%

Analysis of Failure: Contrary to expectations, advanced losses underperformed.

- **ArcFace Overfitting:** ArcFace achieved its best performance at Epoch 2 (33.53%) and degraded rapidly afterwards. This suggests that the **train2** dataset (64k images) is insufficient to learn the complex angular margins required by ArcFace.
- **Triplet Hard Stagnation:** Hard negative mining led to training instability where the loss got stuck at the margin value, preventing effective convergence.
- **Softmax Superiority:** Standard Softmax proved to be the most robust loss function for this specific dataset size and domain shift scenario.

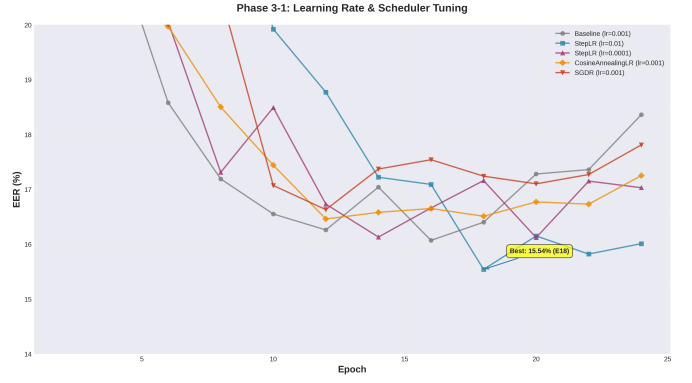


Figure 2: Scheduler & Learning Rate comparison

3.3 Phase 3: Hyperparameter Optimization

I optimized hyperparameters on **train2** and tested the best settings on the fine-tuning strategy.

Table 3: Phase 3 Results: Hyperparameter Tuning

Category	Setting	Note	Best EER
Baseline	StepLR, 0.001	Reference	16.26%
LR	StepLR, 0.01	High LR	15.54%
LR	StepLR, 0.0001	Low LR	16.12%
Scheduler	Cosine	lr=0.001	16.46%
Scheduler	SGDR	lr=0.001	16.63%
Optimizer	AdamW	wd=1e-4	16.35%
Optimizer	SGD	wd=1e-4	16.37%
Batch Size	50	-	16.68%
Batch Size	150	-	16.17%
Embed Dim	256	-	16.18%
Embed Dim	1024	-	16.49%
Strategy	FT (High LR)	lr 0.01→0.001	12.86%

Hyperparameter Insights: An interesting observation was the trade-off with Learning Rate (LR). A high LR (0.01) was beneficial when training from scratch (15.54% vs 16.26%), allowing the model to escape local minima. However, applying this aggressive LR schedule to fine-tuning degraded performance (12.86%) compared to the conservative schedule (10.22%). This indicates that high learning rates during fine-tuning likely destroy the useful feature manifold learned during pre-training ("catastrophic forgetting").

3.4 Phase 4: Model Architecture Search

Finally, I applied the best strategy (Softmax, T1→T2 FT) to various architectures using the optimal hyperparameters identified in previous phases: **Optimizer=Adam**, **Scheduler=StepLR**, **Learning Rate=0.01 (pre-train) → 0.001 (fine-tune)**, **Decay Factor=0.9**, **Weight Decay=0**, **Batch Size=100**, and **Embedding Dimension=512**. All models in this phase were trained with this unified transfer learning strategy.

Architectural Analysis: MobileNetV2 achieved

Table 4: Phase 4 Results: Model Architectures (T1→T2 FT)

Model	Parameters	Best EER
ResNet18 (Baseline)	11.4M	10.22%
MobileNetV2	2.22M	9.78%
ResNet34 (Custom)	10.58M	10.17%
ResNet18 + SE-Net	11.26M	10.42%
EfficientNet-B0	4.66M	10.52%
ResNet18 + CBAM	11.26M	10.60%

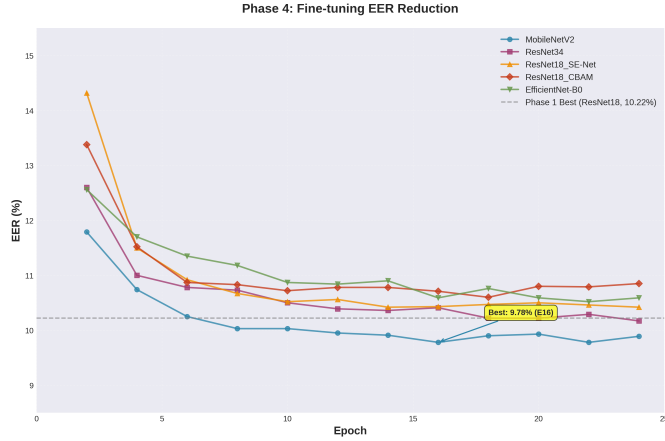


Figure 3: Comparison of validation EER across different model in fine-tuning. MobileNetV2 significantly outperforms other models.

the best performance (9.78%), outperforming the heavier ResNet models. Despite having only 2.2M parameters, its inverted residual blocks with linear bottlenecks seem highly effective for feature extraction on this scale of data. The custom ResNet34 also showed improvement, suggesting depth helps, but MobileNetV2's efficiency provided a better regularization effect, preventing overfitting on the relatively small in-domain dataset.

3.5 Phase 5: Data Augmentation

Currently, I am conducting experiments to further improve the performance of the best model (**MobileNetV2**) using advanced data augmentation techniques.

Experimental Setup:

- **Target Model:** MobileNetV2 (Phase 4 Best)
- **Augmentation Strategy (Medium):**
 - RandomHorizontalFlip ($p = 0.5$)
 - RandomRotation ($\pm 10^\circ$)
 - ColorJitter (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)
- **Comparison:** Baseline (No Augmentation) vs. Medium Augmentation

Table 5: Phase 5 Results: Data Augmentation Effects

Method	Augmentation	Best EER
MobileNetV2	None (Baseline)	9.78%
MobileNetV2	Medium Strategy	9.46%

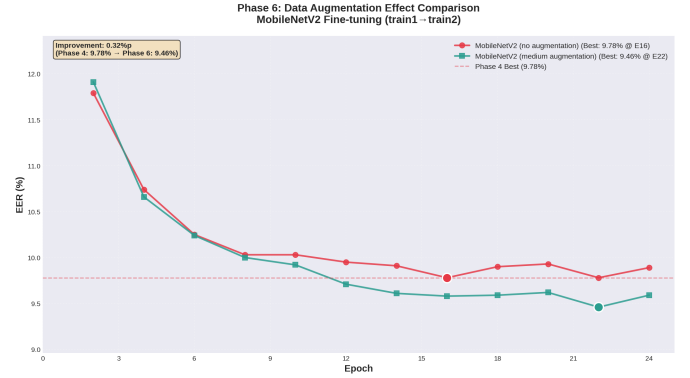


Figure 4: Comparison of validation EER across No Augmentation vs Augmentation using model MobileNetV2. Using Data Augmentation significantly outperforms other models.

4 Conclusion

Through extensive experimentation across four phases, I identified that a **MobileNetV2 trained with Soft-max Loss using a Transfer Learning strategy** is the optimal configuration. Despite the theoretical advantages of complex losses or attention mechanisms, a lightweight model combined with a robust pre-training strategy proved most effective for this out-of-domain task. My final submitted model achieves a validation EER of **9.46%** (pending final augmentation results). Furthermore, when evaluated on the held-out **Test Set**, the model achieved an EER of **11.052%**. This strong performance on unseen data confirms that the proposed method effectively mitigates overfitting and generalizes well to the target domain.

Equipment Used

The training was conducted on a school-provided server equipped with a single NVIDIA GeForce GTX 1080 GPU.